

AI HIRING BOT

Complete System Design & Implementation Blueprint

Powered by Ollama (llama3.1:8b + mistral:7b) — 100% Local, No Cloud LLM

Version 2.0	Date June 2026	LLM Engine Ollama (local)	Status Blueprint
-----------------------	--------------------------	-------------------------------------	----------------------------

Table of Contents

Table of Contents.....	2
1. System Overview	4
1.1 Why Ollama?	4
1.2 Hardware Fit Analysis.....	4
1.3 Model Assignment.....	4
1.4 High-Level Architecture	5
1.5 Full Tech Stack.....	5
2. Ollama Setup & Configuration	7
2.1 Installation	7
2.2 Ollama Python Client.....	7
2.3 Ollama Client Wrapper (shared utility)	7
2.4 GPU Verification.....	8
3. The 6-Layer AI Pipeline.....	9
Layer 1 — Resume Ingestion	9
Layer 2 — Profile Extraction (llama3.1:8b)	9
Layer 3 — JD Matching & Scoring (llama3.1:8b)	10
Layer 4 — Question Generation (llama3.1:8b).....	11
Layer 5 — Voice Interview Engine (mistral:7b)	12
Layer 6 — Report Generation (llama3.1:8b)	12
4. Task Queue — Preventing GPU Conflicts.....	14
4.1 Queue Architecture.....	14
5. Voice System — Twilio + Deepgram + ElevenLabs.....	15
5.1 Call Initiation.....	15
5.2 Webhook Handlers (FastAPI)	15
5.3 ElevenLabs TTS.....	17
5.4 Redis Call State Manager	17
6. HR Admin Interface — All Features	19
6.1 Overview Dashboard	19
6.2 Upload Module	19
6.3 Candidate List	19
6.4 Scheduling Module	19
6.5 Interview Logs & Transcript Viewer.....	19
6.6 Question Bank Editor.....	20
6.7 Settings	20
7. Candidate Interface — All Features	21

Step 1 — Apply & Upload	21
Step 2 — Application Status Timeline	21
Step 3 — Pick a Call Time Slot	21
Step 4 — Pre-Call Guidance & Consent	21
Step 5 — Post-Call Status	21
8. Database Schema (PostgreSQL)	22
candidates	22
jobs	22
screening_calls	22
score_reports	23
9. Project Folder Structure	24
10. Environment Setup & Dependencies	26
10.1 Python Dependencies (requirements.txt)	26
10.2 Environment Variables (.env)	26
10.3 Legal Compliance (India)	27
11. Implementation Roadmap	28
Phase 1 — Ollama + Core Backend (Week 1–2)	28
Phase 2 — Question Gen + Voice Engine (Week 3–4)	28
Phase 3 — Scoring + Reports (Week 5)	28
Phase 4 — Notifications + Scheduling (Week 6)	28
Phase 5 — HR Dashboard (Week 7–8)	28
Phase 6 — Candidate Portal (Week 9)	29
Phase 7 — Testing & Hardening (Week 10)	29
Phase 8 — Deployment (Week 11–12)	29
12. Quick Start — First Call in Under an Hour	30

1. System Overview

The AI Hiring Bot is a fully automated, voice-driven recruitment screening system that runs entirely on your local machine using Ollama as the LLM engine. No cloud AI APIs are used — all intelligence is powered by locally hosted open-source models, giving you full privacy, zero API costs, and complete control over the system.

1.1 Why Ollama?

- **100% local — candidate data never leaves your machine or network**
 - Resumes, transcripts, scores all processed on-device
 - Critical for data privacy and DPDP Act 2023 compliance
- **Zero per-call LLM cost — no Gemini / OpenAI API bills**
 - Unlimited calls, unlimited candidates, flat infrastructure cost
- **Full control over model selection and prompting**
 - Swap models without changing application code
- Works offline — no internet dependency for AI inference

1.2 Hardware Fit Analysis

Your Laptop Specs vs Requirements

CPU: Intel Core i9 — excellent for CPU-side tasks (parsing, routing, scheduling)

RAM: 32 GB — sufficient to run Ollama models + FastAPI + PostgreSQL + Redis simultaneously

GPU: RTX 4050 6 GB VRAM — both llama3.1:8b and mistral:7b fit fully in VRAM (no CPU offloading)

Storage: 1 TB — plenty for model files (~5 GB each), resumes, recordings, and DB

Expected LLM response time: 1–3 seconds per response on GPU (acceptable for voice loop)

Concurrency limit: 1 active call at a time (GPU handles one model at a time)

Recommendation: Queue analysis tasks (Layers 2-4, 6) to run between calls, not during

1.3 Model Assignment

Layer	Task	Model	Why
Layer 2	Profile extraction from resume	llama3.1:8b	Strong JSON output, high accuracy for structured extraction
Layer 3	JD matching & scoring	llama3.1:8b	Reliable reasoning for multi-dimension scoring
Layer 4	Question generation	llama3.1:8b	Best instruction-following for role-specific Q generation
Layer 5	Live interview responses	mistral:7b	Fastest response time — critical for real-time voice loop
Layer 6	Score report generation	llama3.1:8b	Deep evaluation and structured report output

VRAM Strategy

mistral:7b is loaded during active calls (Layer 5) — ~4.1 GB VRAM

llama3.1:8b is loaded for analysis tasks (Layers 2,3,4,6) — ~5.5 GB VRAM

Both models cannot run simultaneously on 6 GB VRAM

Solution: Ollama auto-swaps models on request (adds ~2-3s on first swap, cached after)

Implementation: run analysis tasks in a background queue, never during a live call

1.4 High-Level Architecture

Layer	Name	Purpose	Model / Tech
Layer 1	Resume Ingestion	Parse PDF/DOCX to raw text	PyMuPDF, python-docx
Layer 2	Profile Extraction	Extract structured JSON profile	Ollama — llama3.1:8b
Layer 3	JD Matching & Scoring	Score candidate vs job description	Ollama — llama3.1:8b
Layer 4	Question Generation	Generate 8-12 personalised questions	Ollama — llama3.1:8b
Layer 5	Voice Interview Engine	Live call: STT → Ollama → TTS loop	Ollama — mistral:7b + Twilio + ElevenLabs
Layer 6	Report Generation	Score card + hire recommendation	Ollama — llama3.1:8b

1.5 Full Tech Stack

Component	Technology	Purpose
LLM Engine	Ollama (local)	Runs all AI models on your hardware
Analysis Model	llama3.1:8b	Layers 2, 3, 4, 6 — accuracy-focused tasks
Interview Model	mistral:7b	Layer 5 — speed-focused live conversation
Backend API	FastAPI (Python)	Core server, routing, auth, webhooks
Database	PostgreSQL	Candidates, jobs, calls, transcripts
Cache / State	Redis	Live call state, task queue, session management
Resume Parsing	PyMuPDF + python-docx	Extract raw text from uploaded files
Voice Calls	Twilio Voice API	Outbound dialing, speech input, webhooks
Text-to-Speech	ElevenLabs API	Natural-sounding HR voice output
Speech-to-Text	Deepgram (real-time)	Accurate live call transcription
Task Queue	Celery + Redis	Background LLM jobs, prevents GPU conflicts
Frontend	React + Tailwind CSS	HR dashboard + Candidate portal
File Storage	Local disk / MinIO	Resumes, recordings (no S3 needed)
Notifications	Twilio SMS + SendGrid	Email + SMS to candidates and HR

Dev tunnel	ngrok	Expose local webhooks to Twilio during dev
Deployment	Docker + Nginx	Containerised production setup

2. Ollama Setup & Configuration

2.1 Installation

```
# Step 1: Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Step 2: Pull both models
ollama pull llama3.1:8b      # ~4.7 GB download — analysis model
ollama pull mistral:7b       # ~4.1 GB download — interview model

# Step 3: Verify models are available
ollama list

# Step 4: Test llama3.1:8b
ollama run llama3.1:8b 'Return a JSON object with key: test, value: success'

# Step 5: Test mistral:7b
ollama run mistral:7b 'Say hello in one sentence'

# Step 6: Start Ollama as a background service
ollama serve # Runs on http://localhost:11434
```

2.2 Ollama Python Client

Ollama exposes a simple REST API on localhost:11434. We use the official Python library for clean integration:

```
pip install ollama

# Basic usage
import ollama

response = ollama.chat(
    model='llama3.1:8b',
    messages=[{'role': 'user', 'content': 'Your prompt here'}]
)
print(response['message']['content'])
```

2.3 Ollama Client Wrapper (shared utility)

A single wrapper used by all layers — handles model selection, retries, JSON parsing, and timeout:

```
# backend/services/ollama_client.py
import ollama
import json
```

```
import re
from typing import Optional

ANALYSIS_MODEL = 'llama3.1:8b'    # Layers 2, 3, 4, 6
INTERVIEW_MODEL = 'mistral:7b'   # Layer 5 (live call)

def ollama_chat(prompt: str, model: str = ANALYSIS_MODEL,
                expect_json: bool = False) -> str:
    """Send a prompt to Ollama and return the response text."""
    try:
        response = ollama.chat(
            model=model,
            messages=[{'role': 'user', 'content': prompt}],
            options={
                'temperature': 0.3,    # Low temp for consistent structured output
                'num_predict': 2048,    # Max tokens per response
            }
        )
        text = response['message']['content'].strip()
        if expect_json:
            return extract_json(text)
        return text
    except Exception as e:
        raise RuntimeError(f'Ollama error: {e}')

def extract_json(text: str) -> dict:
    """Strip markdown fences and parse JSON from LLM output."""
    # Remove ```json ... ``` fences if present
    clean = re.sub(r'```(?::json)?\n?', '', text).replace('```', '').strip()
    return json.loads(clean)
```

2.4 GPU Verification

```
# Confirm Ollama is using your RTX 4050
ollama run llama3.1:8b 'hello'

# In another terminal, check GPU usage
nvidia-smi

# You should see ollama runner process using ~5.5 GB VRAM
# If not using GPU, set this env var:
export CUDA_VISIBLE_DEVICES=0
ollama serve
```


3. The 6-Layer AI Pipeline

Layer 1 — Resume Ingestion

No LLM involved. Accepts PDF, DOCX, or plain text. Extracts clean raw text for downstream layers.

```
# backend/services/resume_parser.py
import fitz # PyMuPDF
from docx import Document as DocxDoc

def extract_resume_text(file_path: str) -> str:
    if file_path.endswith('.pdf'):
        doc = fitz.open(file_path)
        return '\n'.join(page.get_text() for page in doc)
    elif file_path.endswith('.docx'):
        doc = DocxDoc(file_path)
        return '\n'.join(p.text for p in doc.paragraphs if p.text.strip())
    else:
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
```

Layer 2 — Profile Extraction (llama3.1:8b)

Sends resume text to llama3.1:8b and extracts a structured JSON candidate profile. This profile is stored in the DB and used by all subsequent layers.

```
# backend/services/profile_extractor.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL

def extract_profile(resume_text: str) -> dict:
    prompt = f'''
You are an expert HR analyst. Extract a structured JSON profile from the resume
below.
Return ONLY valid JSON — no explanation, no markdown fences.

Required JSON schema:
{{
  "name": string,
  "email": string,
  "phone": string,
  "skills": [list of technical and soft skills],
  "experience_years": integer,
  "roles": [{{"title": str, "company": str, "duration": str, "responsibilities":
[str]}]},
  "education": [{{"degree": str, "institution": str, "year": str}},
  "projects": [{{"name": str, "tech_stack": [str], "description": str}},
  "certifications": [str],
  "employment_gaps": [{{"period": str, "duration": str}},
```

```

    "strengths": [top 3 strengths as strings],
    "red_flags": [list of concerns as strings],
    "languages": [spoken or programming languages]
  }}

Resume:
{resume_text}
'''

    return ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)

```

Layer 3 — JD Matching & Scoring (llama3.1:8b)

Scores the candidate across five dimensions against the job description. Returns a structured score object with reasoning.

Dimension	Weight	What is evaluated
Skills match	30%	Required vs actual technical skills overlap
Experience	25%	Years of experience, seniority, domain fit
Education	15%	Degree level, institution relevance, field match
Project relevance	20%	Past projects matching role requirements
Red flags	10%	Employment gaps, frequent job switching, inconsistencies

```

# backend/services/jd_matcher.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL

def score_candidate(profile: dict, jd_text: str) -> dict:
    prompt = f'''
You are a senior HR recruiter. Score this candidate against the job description.
Return ONLY valid JSON with this exact schema:
{{
  "skills_score": 0-100,
  "experience_score": 0-100,
  "education_score": 0-100,
  "project_score": 0-100,
  "red_flag_penalty": 0-100 (100 = no penalty),
  "overall_score": weighted average 0-100,
  "matched_skills": [list of matching skills],
  "missing_skills": [list of required skills candidate lacks],
  "reasoning": "2-3 sentence summary of the match"
}}

Candidate Profile: {profile}

Job Description: {jd_text}
'''

    return ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)

```

```
AUTO_REJECT_THRESHOLD = 40

def should_proceed(score: dict) -> str:
    overall = score.get('overall_score', 0)
    if overall < AUTO_REJECT_THRESHOLD:
        return 'reject'
    elif overall < 60:
        return 'review' # HR manual review before scheduling
    return 'proceed'
```

Layer 4 — Question Generation (llama3.1:8b)

Generates a personalised set of 8-12 questions based on the specific candidate profile and JD. Questions are ordered for natural conversation flow.

```
# backend/services/question_gen.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL

def generate_questions(profile: dict, jd_text: str,
                      count: int = 10) -> list[dict]:
    prompt = f'''
You are an expert technical interviewer. Generate {count} screening questions
for this specific candidate applying for this role.

Return ONLY a valid JSON array of question objects:
[
  {{
    "id": 1,
    "type": "warmup|technical|behavioral|resume_probe|situational|closing",
    "question": "The full question text",
    "ideal_answer_points": ["key points to listen for"],
    "follow_up": "Optional follow-up if answer is vague"
  }}
]

Guidelines:
- Start with 1-2 warmup questions
- Include questions about specific skills gaps you identified
- Probe any employment gaps or short tenures in their history
- End with a closing question inviting their questions
- Make questions feel conversational, not interrogative

Candidate Profile: {profile}
Job Description: {jd_text}
'''
    return ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)
```

Layer 5 — Voice Interview Engine (mistral:7b)

The real-time interview loop. mistral:7b is used here for its speed — every second of silence on a phone call feels awkward. The model receives the full conversation history on each turn and decides the next response.

```
# backend/services/interview_engine.py
from .ollama_client import ollama_chat, INTERVIEW_MODEL

def generate_next_response(call_state: dict) -> str:
    """
    Given the full call state (candidate info, questions, transcript so far),
    generate the bot's next spoken response using mistral:7b.
    """
    transcript_text = '\n'.join(
        f"{t['speaker'].upper()}: {t['text']}"
        for t in call_state['transcript']
    )
    current_q = call_state['questions'][call_state['current_q_index']]

    prompt = f'''
You are Aria, a professional AI HR assistant conducting a phone screening call.
You are friendly, concise, and professional. Keep responses SHORT (2-4 sentences
max).
You are currently in the {call_state['phase']} phase of the call.

Candidate: {call_state['candidate_name']}
Role: {call_state['role']}
Current question to ask: {current_q['question']}
Follow-up hint: {current_q.get('follow_up', 'N/A')}

Conversation so far:
{transcript_text}

Instructions:
- Acknowledge the candidate's last answer briefly (1 sentence)
- If the answer was too vague, use the follow-up before moving on
- Otherwise, transition naturally to the next question
- Never repeat a question already asked
- Speak naturally as if on a real phone call

Respond with ONLY what Aria should say next. No labels, no quotes.
'''
    return ollama_chat(prompt, model=INTERVIEW_MODEL, expect_json=False)
```

Layer 6 — Report Generation (llama3.1:8b)

After the call ends, the full transcript is evaluated by llama3.1:8b to produce a detailed score card and hiring recommendation.

```
# backend/services/report_gen.py
from .ollama_client import ollama_chat, ANALYSIS_MODEL

def generate_report(call_state: dict, profile: dict) -> dict:
    transcript_text = '\n'.join(
        f"{t['speaker'].upper(): {t['text']}}"
        for t in call_state['transcript']
    )
    prompt = f'''
You are a senior HR evaluator. Analyse this screening call transcript and
produce a detailed candidate evaluation report.
Return ONLY valid JSON with this schema:
{{
  "overall_score": 0-100,
  "skills_score": 0-100,
  "experience_score": 0-100,
  "communication_score": 0-100,
  "culture_fit_score": 0-100,
  "confidence_score": 0-100,
  "recommendation": "HIRE|SHORTLIST|HOLD|REJECT",
  "reasoning": "3-4 sentence detailed justification",
  "strengths": ["list of observed strengths"],
  "red_flags": ["list of concerns from the call"],
  "next_round_questions": ["suggested questions for round 2"]
}}

Candidate: {call_state['candidate_name']} | Role: {call_state['role']}
Pre-call JD match score: {call_state['match_score']}%

Full transcript:
{transcript_text}
'''
    return ollama_chat(prompt, model=ANALYSIS_MODEL, expect_json=True)
```

4. Task Queue — Preventing GPU Conflicts

Since both Ollama models cannot run simultaneously on 6 GB VRAM, all heavy LLM tasks (Layers 2, 3, 4, 6) are processed through a Celery task queue backed by Redis. This ensures analysis jobs never interrupt an active voice call.

4.1 Queue Architecture

Queue	Workers	Tasks	Priority
call_queue	1 worker	Layer 5 live responses (mistral:7b)	HIGHEST — never blocked
analysis_queue	1 worker	Layers 2, 3, 4, 6 (llama3.1:8b)	NORMAL — runs between calls
notification_queue	2 workers	Email, SMS sending	LOW — no GPU needed

```
# backend/celery_app.py
from celery import Celery

celery = Celery(
    'hiring_bot',
    broker='redis://localhost:6379/0',
    backend='redis://localhost:6379/1'
)

celery.conf.task_routes = {
    'tasks.run_profile_extraction': {'queue': 'analysis_queue'},
    'tasks.run_jd_scoring':         {'queue': 'analysis_queue'},
    'tasks.run_question_gen':       {'queue': 'analysis_queue'},
    'tasks.run_report_gen':         {'queue': 'analysis_queue'},
    'tasks.send_email':             {'queue': 'notification_queue'},
    'tasks.send_sms':               {'queue': 'notification_queue'},
}

# Start workers (run in separate terminals):
# celery -A backend.celery_app worker -Q analysis_queue -c 1 --loglevel=info
# celery -A backend.celery_app worker -Q notification_queue -c 2 --loglevel=info
```

5. Voice System — Twilio + Deepgram + ElevenLabs

5.1 Call Initiation

```
# backend/voice/twilio_client.py
from twilio.rest import Client
import os

client = Client(os.getenv('TWILIO_ACCOUNT_SID'), os.getenv('TWILIO_AUTH_TOKEN'))

def dial_candidate(phone: str, call_sid_key: str) -> str:
    call = client.calls.create(
        to=phone,
        from_=os.getenv('TWILIO_PHONE_NUMBER'),
        url=f"{os.getenv('WEBHOOK_BASE_URL')}/voice/start",
        status_callback=f"{os.getenv('WEBHOOK_BASE_URL')}/voice/status",
        status_callback_method='POST',
        record=True,

        recording_status_callback=f"{os.getenv('WEBHOOK_BASE_URL')}/voice/recording"
    )
    return call.sid
```

5.2 Webhook Handlers (FastAPI)

```
# backend/routers/voice.py
from fastapi import APIRouter, Form, Response
from twilio.twiml.voice_response import VoiceResponse, Gather
from ..voice.tts import text_to_speech_url
from ..voice.call_state import get_state, save_state
from ..services.interview_engine import generate_next_response
import json

router = APIRouter(prefix='/voice')

@router.post('/start')
async def call_start(CallSid: str = Form(...)):
    state = get_state(CallSid)
    intro = (
        f"Hello, am I speaking with {state['candidate_name']}? "
        f"Great! I'm Aria, an AI HR assistant calling on behalf of "
        f"{state['company']}. "
        f"I'll be asking around {len(state['questions'])} screening questions "
        f"for the {state['role']} role — this should take about 12-15 minutes. "
        f>Please note you are speaking with an AI assistant. "
        f"Are you ready to begin?"
    )
    state['transcript'].append({'speaker': 'bot', 'text': intro})
    state['phase'] = 'intro'
    save_state(CallSid, state)
    audio_url = text_to_speech_url(intro)
```

```

vr = VoiceResponse()
vr.play(audio_url)
vr.gather(input='speech', action='/voice/respond',
          speechTimeout='3', language='en-IN')
return Response(str(vr), media_type='application/xml')

@router.post('/respond')
async def call_respond(
    CallSid: str = Form(...),
    SpeechResult: str = Form(''),
):
    state = get_state(CallSid)
    # Save candidate answer
    if SpeechResult:
        state['transcript'].append({'speaker': 'candidate', 'text': SpeechResult})
    else:
        state['transcript'].append({'speaker': 'candidate', 'text': '[silence]'})
        state['silence_count'] = state.get('silence_count', 0) + 1
    # Check if interview is complete
    if state['current_q_index'] >= len(state['questions']):
        return await end_call(CallSid, state)
    # Generate next bot response via mistral:7b
    next_response = generate_next_response(state)
    state['transcript'].append({'speaker': 'bot', 'text': next_response})
    # Advance question index if not a follow-up
    if state.get('asked_followup') or state.get('silence_count', 0) == 0:
        state['current_q_index'] += 1
        state['asked_followup'] = False
        state['silence_count'] = 0
    save_state(CallSid, state)
    audio_url = text_to_speech_url(next_response)
    vr = VoiceResponse()
    vr.play(audio_url)
    vr.gather(input='speech', action='/voice/respond',
              speechTimeout='4', language='en-IN')
    return Response(str(vr), media_type='application/xml')

async def end_call(call_sid: str, state: dict):
    closing = (
        f"Thank you so much for your time, {state['candidate_name']}. "
        "That's all the questions I have. HR will review the call and "
        "get back to you within 2 business days. Have a great day!"
    )
    state['transcript'].append({'speaker': 'bot', 'text': closing})
    state['phase'] = 'completed'
    save_state(call_sid, state)
    # Trigger report generation as background task
    from ..tasks import run_report_gen
    run_report_gen.delay(call_sid)
    vr = VoiceResponse()
    audio_url = text_to_speech_url(closing)
    vr.play(audio_url)

```



```
vr.hangup()
return Response(str(vr), media_type='application/xml')
```

5.3 ElevenLabs TTS

```
# backend/voice/tts.py
import requests, os, hashlib
from pathlib import Path

AUDIO_CACHE_DIR = Path('audio_cache')
AUDIO_CACHE_DIR.mkdir(exist_ok=True)

def text_to_speech_url(text: str) -> str:
    """Convert text to speech via ElevenLabs, cache audio files."""
    cache_key = hashlib.md5(text.encode()).hexdigest()
    cache_path = AUDIO_CACHE_DIR / f'{cache_key}.mp3'
    if cache_path.exists():
        return f"{os.getenv('WEBHOOK_BASE_URL')}/audio/{cache_key}.mp3"
    response = requests.post(
        f"https://api.elevenlabs.io/v1/text-to-speech/{os.getenv('ELEVENLABS_VOICE_ID')}",
        headers={'xi-api-key': os.getenv('ELEVENLABS_API_KEY')},
        json={'text': text, 'model_id': 'eleven_turbo_v2',
            'voice_settings': {'stability': 0.5, 'similarity_boost': 0.75}}
    )
    cache_path.write_bytes(response.content)
    return f"{os.getenv('WEBHOOK_BASE_URL')}/audio/{cache_key}.mp3"
```

5.4 Redis Call State Manager

```
# backend/voice/call_state.py
import redis, json
from datetime import datetime

r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)
CALL_TTL = 7200 # 2 hours

def init_state(call_sid: str, candidate: dict, questions: list) -> dict:
    state = {
        'call_sid': call_sid,
        'candidate_name': candidate['name'],
        'company': candidate['company'],
        'role': candidate['role'],
        'match_score': candidate['match_score'],
        'questions': questions,
        'current_q_index': 0,
        'transcript': [],
        'phase': 'init',
        'silence_count': 0,
        'asked_followup': False,
        'started_at': datetime.utcnow().isoformat()
```

```
    }
    r.setex(f'call:{call_sid}', CALL_TTL, json.dumps(state))
    return state

def get_state(call_sid: str) -> dict:
    data = r.get(f'call:{call_sid}')
    if not data:
        raise ValueError(f'No call state found for {call_sid}')
    return json.loads(data)

def save_state(call_sid: str, state: dict):
    r.setex(f'call:{call_sid}', CALL_TTL, json.dumps(state))
```

6. HR Admin Interface — All Features

6.1 Overview Dashboard

- Live stats: total candidates, scheduled calls, completed, avg match score
- Today's call schedule with name, role, call time, JD match %, status badge
- Weekly pipeline funnel: applied → analyzed → scheduled → screened → shortlisted
- Recent activity feed: last 10 events across all candidates
- Quick-action buttons: add candidate, view pending reviews, export all reports

6.2 Upload Module

- Resume upload: PDF or DOCX drag-and-drop, or file browser
- Bulk upload: ZIP file containing multiple resumes — auto-processed in queue
- Job Description: upload PDF, paste text, or select from internal JD library
- Candidate details form: name, phone, email, role, source (LinkedIn / referral / portal)
- On submit: Layers 1-3 run in background via Celery — match score shown within ~15 seconds
- HR can review and correct the AI-extracted profile before proceeding
- Auto-reject notification sent immediately for candidates below threshold

6.3 Candidate List

- Searchable, filterable, sortable table of all candidates
- Filters: role, status, match score range, date added, source
- Bulk actions: assign screening date, export CSV, send bulk email, bulk reject
- Click any candidate to open full profile with extracted skills, experience, red flags

6.4 Scheduling Module

- HR selects a screening DATE per candidate from a calendar
- System sends candidate email + SMS with link to pick their time slot
- Weekly calendar view showing all scheduled calls with status
- Manual time override: HR can force a specific date and time
- Re-schedule: reset a candidate's slot and trigger new notification
- Retry config: auto-retry on no-answer (default: 3 retries, 30 min apart)

6.5 Interview Logs & Transcript Viewer

- Full list of completed calls with search and date filter
- Per-call view: score card with 6 dimension bars, AI recommendation, full reasoning
- Full verbatim Q&A transcript with timestamps
- Audio playback of call recording
- Red flags and strengths highlighted inline in transcript

- HR can add manual notes to any transcript
- HR override: approve / reject / hold with a required reason field
- Export transcript + score card as PDF

6.6 Question Bank Editor

- View, edit, add, delete questions per role
- Questions tagged by: role, type (technical / behavioral / situational), difficulty
- Pin questions that must always appear for a given role
- Set total question count and type distribution per role

6.7 Settings

- Ollama model selector: choose which model for analysis vs interview
- Ollama base URL: configure if Ollama runs on a different machine
- Bot intro script: customise the opening greeting
- Scoring weights: adjust per-dimension weights (must sum to 100%)
- Auto-reject threshold: minimum match score to proceed (default: 40%)
- Retry settings: number of attempts and interval between retries
- Language: English or Hindi (affects TTS voice and LLM prompt language)
- API keys: Twilio, ElevenLabs, Deepgram, SendGrid
- Call recording: enable or disable
- AI disclosure: toggle explicit AI identification on call (cannot be fully disabled — legal)

7. Candidate Interface — All Features

Step 1 — Apply & Upload

- Upload resume (PDF or DOCX)
- Fill in: full name, phone number (E.164), email
- Optionally attach or paste the job description
- Submit → triggers background Celery tasks for Layers 1-3
- Confirmation screen: 'Application received — HR will notify you soon'

Step 2 — Application Status Timeline

- Visual progress tracker: resume submitted → HR reviewed → date assigned → time confirmed → call done → result
- Each step shows date and additional info (e.g. match score if HR enables visibility)
- Email + SMS sent on every stage transition

Step 3 — Pick a Call Time Slot

- HR assigns the DATE; candidate picks their preferred TIME
- Available slots shown as a grid (HR configures available hours)
- Taken slots grayed out in real time
- On confirm: SMS + email with date/time + .ics calendar invite
- Reminder SMS 30 minutes before the call
- Reschedule: change time up to 2 hours before the call

Step 4 — Pre-Call Guidance & Consent

- What to expect: intro → warmup → screening questions → candidate Q&A → closing
- Prep tips: quiet space, phone charged, water nearby, have resume handy
- FAQ: Is this AI? Is it recorded? How long? What happens after?
- AI consent checkbox — must be ticked before call is allowed to proceed
- Consent timestamp logged in DB for compliance

Step 5 — Post-Call Status

- After call: 'Screening complete — HR will contact you within 2 business days'
- If shortlisted: email with next steps
- If rejected: polite rejection email with optional feedback from HR

8. Database Schema (PostgreSQL)

candidates

Column	Type	Description
id	UUID PK	Unique candidate ID
name	VARCHAR	Full name
email	VARCHAR UNIQUE	Email address
phone	VARCHAR	Phone in E.164 format (+91XXXXXXXXXX)
resume_url	TEXT	Local or MinIO path to resume file
jd_id	UUID FK	Linked job description
profile_json	JSONB	Layer 2 extracted profile (llama3.1:8b output)
match_score	INTEGER	Layer 3 JD match score 0-100
match_details	JSONB	Per-dimension scores and reasoning
status	ENUM	pending/scheduled/in_call/completed/rejected
source	VARCHAR	LinkedIn/referral/portal/other
consent_given	BOOLEAN	AI call consent recorded
consent_at	TIMESTAMP	When consent was given
created_at	TIMESTAMP	Application timestamp

jobs

Column	Type	Description
id	UUID PK	Unique job ID
title	VARCHAR	Job title
department	VARCHAR	Department name
jd_text	TEXT	Full job description text
required_skills	JSONB	Parsed required skills list
min_experience	INTEGER	Minimum years required
auto_reject_threshold	INTEGER	Min match score (default: 40)
question_count	INTEGER	Questions to ask (default: 10)
pinned_questions	JSONB	Questions always included
ollama_model_override	VARCHAR	Optional per-job model override

screening_calls

Column	Type	Description
id	UUID PK	Call record ID

candidate_id	UUID FK	Linked candidate
twilio_call_sid	VARCHAR	Twilio call SID
scheduled_date	DATE	Date set by HR
scheduled_time	TIME	Time slot chosen by candidate
call_started_at	TIMESTAMP	Actual call start
call_ended_at	TIMESTAMP	Actual call end
duration_seconds	INTEGER	Call length in seconds
recording_url	TEXT	Twilio recording URL
transcript	JSONB	Full Q&A [{speaker, text, timestamp}]
retry_count	INTEGER	Number of retry attempts made
interview_model	VARCHAR	Ollama model used (mistral:7b)
status	ENUM	pending/dialing/in_progress/completed/no_answer/failed

score_reports

Column	Type	Description
id	UUID PK	Report ID
call_id	UUID FK	Linked screening call
overall_score	INTEGER	0-100 overall score
skills_score	INTEGER	Technical skills score
experience_score	INTEGER	Experience relevance score
communication_score	INTEGER	Communication clarity score
culture_fit_score	INTEGER	Culture fit score
confidence_score	INTEGER	Confidence and engagement score
ai_recommendation	ENUM	HIRE/SHORTLIST/HOLD/REJECT
ai_reasoning	TEXT	LLM reasoning for recommendation
red_flags	JSONB	Identified red flags list
strengths	JSONB	Identified strengths list
next_round_questions	JSONB	Suggested round 2 questions
report_model	VARCHAR	Ollama model used (llama3.1:8b)
hr_override	ENUM	HR manual override (nullable)
hr_notes	TEXT	HR additional comments

9. Project Folder Structure

```

hiring-bot/
├── backend/                                # FastAPI application
│   ├── main.py                            # App entry point + router registration
│   ├── config.py                          # Settings via pydantic-settings
│   ├── database.py                        # PostgreSQL + SQLAlchemy async setup
│   ├── redis_client.py                    # Redis connection
│   ├── celery_app.py                      # Celery + task queue config
│   ├── tasks.py                           # Celery task definitions
│   ├── routers/
│   │   ├── hr.py                          # HR admin endpoints
│   │   ├── candidate.py                   # Candidate portal endpoints
│   │   ├── voice.py                       # Twilio webhook handlers
│   │   └── reports.py                     # Score report endpoints
│   ├── services/
│   │   ├── ollama_client.py               # Ollama wrapper (llama3.1:8b + mistral:7b)
│   │   ├── resume_parser.py               # Layer 1: PyMuPDF + python-docx
│   │   ├── profile_extractor.py           # Layer 2: llama3.1:8b
│   │   ├── jd_matcher.py                  # Layer 3: llama3.1:8b
│   │   ├── question_gen.py                # Layer 4: llama3.1:8b
│   │   ├── interview_engine.py            # Layer 5: mistral:7b
│   │   └── report_gen.py                  # Layer 6: llama3.1:8b
│   ├── voice/
│   │   ├── twilio_client.py               # Outbound call dialing
│   │   ├── tts.py                         # ElevenLabs TTS with audio caching
│   │   └── call_state.py                  # Redis call state manager
│   ├── notifications/
│   │   ├── email.py                       # SendGrid email
│   │   └── sms.py                         # Twilio SMS
│   ├── models/                            # SQLAlchemy ORM models
│   │   ├── candidate.py
│   │   ├── job.py
│   │   ├── call.py
│   │   └── report.py
│   └── schemas/                           # Pydantic request/response schemas
│       ├── candidate.py
│       ├── job.py
│       └── report.py
├── frontend/
│   ├── hr-dashboard/                       # React app — HR admin
│   │   └── src/pages/
│   │       ├── Overview.jsx
│   │       ├── Upload.jsx
│   │       ├── Candidates.jsx
│   │       ├── Schedule.jsx
│   │       ├── InterviewLogs.jsx
│   │       └── Settings.jsx
│   └── candidate-portal/                   # React app — candidates

```



```
|      └─ src/pages/
|          └─ Apply.jsx
|          └─ Status.jsx
|          └─ PickSlot.jsx
|          └─ PreCall.jsx
|          └─ PostCall.jsx
|
|─ audio_cache/           # Cached TTS audio files (ElevenLabs)
|─ uploads/              # Uploaded resumes
|─ docker-compose.yml
|─ Dockerfile
|─ requirements.txt
|─ alembic/              # DB migrations
|─ .env.example
|─ README.md
```

10. Environment Setup & Dependencies

10.1 Python Dependencies (requirements.txt)

```
# Core framework
fastapi==0.111.0
uvicorn==0.29.0
python-multipart==0.0.9

# Database
sqlalchemy==2.0.30
alembic==1.13.1
asyncpg==0.29.0
redis==5.0.4

# Task queue
celery==5.4.0
flower==2.0.1          # Celery monitoring UI

# Ollama (replaces all cloud LLM SDKs)
ollama==0.2.1

# Resume parsing
PyMuPDF==1.24.3
python-docx==1.1.2

# Voice
twilio==9.0.5
deepgram-sdk==3.2.7
elevenlabs==1.1.3

# Notifications
sendgrid==6.11.0

# Scheduling
apscheduler==3.10.4

# Auth & validation
pydantic==2.7.1
pydantic-settings==2.3.0
python-jose==3.3.0
passlib==1.7.4

# HTTP
httpx==0.27.0
requests==2.31.0
```

10.2 Environment Variables (.env)

```
# — Ollama (local — no API key needed) —————
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_ANALYSIS_MODEL=llama3.1:8b
OLLAMA_INTERVIEW_MODEL=mistral:7b

# — Database —————
DATABASE_URL=postgresql+asyncpg://user:pass@localhost/hiringbot
REDIS_URL=redis://localhost:6379

# — Twilio —————
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_auth_token
TWILIO_PHONE_NUMBER=+1xxxxxxxxxx

# — ElevenLabs —————
ELEVENLABS_API_KEY=your_elevenlabs_key
ELEVENLABS_VOICE_ID=your_voice_id

# — Deepgram —————
DEEPGRAM_API_KEY=your_deepgram_key

# — SendGrid —————
SENDGRID_API_KEY=your_sendgrid_key
FROM_EMAIL=hr@yourcompany.com

# — App —————
SECRET_KEY=your_jwt_secret_key
WEBHOOK_BASE_URL=https://your-ngrok-url.ngrok.io
UPLOAD_DIR=./uploads
AUDIO_CACHE_DIR=./audio_cache
AUTO_REJECT_THRESHOLD=40
CALL_RETRY_COUNT=3
CALL_RETRY_INTERVAL_MINUTES=30
```

10.3 Legal Compliance (India)

Compliance Checklist

- AI Disclosure: Bot must identify itself as AI at call start — non-negotiable
- Recorded Consent: Candidate confirms consent before call proceeds — logged in DB with timestamp
- Call Recording: Inform candidates if recording; store securely; purge after 90 days
- DPDP Act 2023: All personal data stored locally; define retention policy per data category
- Data Access: Candidates can request their data — build a /candidate/my-data endpoint
- Bias Audit: Monthly review of AI scores across demographic groups
- Transparency: Candidates can request reasoning behind rejection decision

11. Implementation Roadmap

Phase 1 — Ollama + Core Backend (Week 1–2)

- Install Ollama, pull llama3.1:8b and mistral:7b, verify GPU usage
- Set up FastAPI project, PostgreSQL, Redis, Celery
- Implement Layer 1: resume ingestion (PDF + DOCX)
- Implement Layer 2: profile extraction via llama3.1:8b — test with 5 real resumes
- Implement Layer 3: JD matching via llama3.1:8b — verify score accuracy
- Build /hr/candidates/upload end-to-end with Celery background tasks

Phase 2 — Question Gen + Voice Engine (Week 3–4)

- Implement Layer 4: question generation via llama3.1:8b
- Set up Twilio account, buy India-capable number
- Build /voice/start and /voice/respond webhook handlers
- Integrate ElevenLabs TTS with audio caching
- Implement Redis call state manager
- Wire mistral:7b into interview engine (Layer 5)
- Test full voice loop end-to-end on ngrok — make a real test call

Phase 3 — Scoring + Reports (Week 5)

- Implement Layer 6: post-call report generation via llama3.1:8b
- Score card calculation and DB storage
- No-answer retry logic with APScheduler
- Edge case handling: silence, drop, Hindi detection

Phase 4 — Notifications + Scheduling (Week 6)

- SendGrid email integration with templates
- Twilio SMS integration
- Candidate time slot selection API
- .ics calendar invite generation
- 30-minute pre-call reminder SMS

Phase 5 — HR Dashboard (Week 7–8)

- React + Tailwind project setup
- Overview dashboard, upload module, candidate list, schedule module
- Interview logs with transcript viewer and score visualisation
- Question bank editor and settings page

Phase 6 — Candidate Portal (Week 9)

- 4-step candidate flow: apply → status → pick slot → pre-call guidance
- Consent screen with timestamp logging
- Post-call status and result notifications

Phase 7 — Testing & Hardening (Week 10)

- End-to-end test with 10+ real calls
- GPU conflict stress test: upload + simultaneous call
- Prompt tuning: adjust llama3.1:8b prompts for better JSON consistency
- Security: JWT auth, rate limiting, input sanitisation
- Ollama timeout handling: graceful fallback if model is slow

Phase 8 — Deployment (Week 11–12)

- Dockerise all services (FastAPI, Celery, PostgreSQL, Redis, Ollama)
- Ollama in Docker with GPU passthrough (--gpus all)
- Set up Nginx reverse proxy with SSL
- Configure Twilio production webhooks (replace ngrok)
- Set up monitoring: Sentry for errors, Flower for Celery, nvidia-smi alerts
- Final UAT with HR team — go live

12. Quick Start — First Call in Under an Hour

1. Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh`
2. Pull models: `ollama pull llama3.1:8b && ollama pull mistral:7b`
3. Start Ollama server: `ollama serve` (keep this terminal open)
4. Clone/create project, install deps: `pip install -r requirements.txt`
5. Set up PostgreSQL and Redis (Docker recommended: `docker compose up -d db redis`)
6. Copy `.env.example` to `.env`, fill in Twilio, ElevenLabs, SendGrid keys
7. Run DB migrations: `alembic upgrade head`
8. Start FastAPI: `uvicorn backend.main:app --reload --port 8000`
9. Start Celery workers (2 terminals):
 - `celery -A backend.celery_app worker -Q analysis_queue -c 1`
 - `celery -A backend.celery_app worker -Q notification_queue -c 2`
10. Install and run ngrok: `ngrok http 8000` — copy the HTTPS URL to `WEBHOOK_BASE_URL` in `.env`
11. Set Twilio Voice webhook to: `https://your-ngrok-url/voice/start`
12. Upload a test resume via POST `/hr/candidates/upload` — verify match score returns
13. Trigger a test call via POST `/hr/candidates/{id}/call-now` — answer your phone!
14. Check Redis call state: `redis-cli get call:{twilio_call_sid}`
15. After call ends, check PostgreSQL `score_reports` table for the generated report

First Call Checklist

Ollama is running (`ollama serve`) and GPU is active (`nvidia-smi` shows usage)

Both models are pulled: `ollama list` shows `llama3.1:8b` and `mistral:7b`

ElevenLabs voice ID is valid — test it at elevenlabs.io/speech-synthesis

Twilio number is Voice-capable and webhook URL is set to ngrok HTTPS URL

Candidate phone is in E.164 format: `+91XXXXXXXXXX`

Redis is running and reachable on `localhost:6379`

PostgreSQL has candidate + call records created before dialing

Audio cache directory exists and is writable

Celery `analysis_queue` worker is running (no GPU conflict during call)